

Prédiction d'ensembles et quantification d'incertitude

MIASHS M1 - Semestre 1

Table des matières

1. Introduction	1
2. Classification Top-K	2
3. Choix automatique de K	4
4. Prédiction conformelle	6
5. Scores de conformité probabilistes	9
6. Application à la régression	11
7. Synthèse et perspectives	15
8. Conclusion	16

1. Introduction

Jusqu'à présent, nous avons vu comment **combiner plusieurs modèles** pour améliorer les performances. Nous abordons maintenant une question complémentaire : au lieu de prédire une **unique** valeur, comment peut-on prédire un **ensemble de valeurs** avec des garanties de couverture ?

Cette approche est cruciale dans les applications critiques où il faut **quantifier l'incertitude** :

- Diagnostic médical : « Ces 3 pathologies sont les plus probables »
- Reconnaissance d'images : « L'objet est soit un chat, soit un chien »
- Finance : « Le prix sera dans cet intervalle avec 95% de confiance »

Paradigme des ensembles de prédiction

Au lieu de chercher $\hat{y} = f(x)$, on cherche un ensemble

$$\mathcal{C}(x) \subset \mathcal{Y}$$

tel que :

$$\mathbb{P}(Y \in \mathcal{C}(X)) \geq 1 - \alpha$$

où α est le niveau de risque accepté (ex: $\alpha = 0.05$ pour 95% de confiance).

2. Classification Top-K

2.1. Principe et motivation

La méthode Top-K est l'approche la plus intuitive pour la classification multi-classe.

Définition 6.1 (Prédiction Top-K)

Soit $\hat{p}(x) = (\hat{p}_1(x), \dots, \hat{p}_C(x))$ les probabilités prédites pour les C classes. La prédiction Top-K retourne :

$$\mathcal{C}_K(x) = \{\text{les } K \text{ classes avec les plus hautes probabilités}\}$$

Formellement, si π est la permutation telle que $\hat{p}_{\pi(1)}(x) \geq \hat{p}_{\pi(2)}(x) \geq \dots \geq \hat{p}_{\pi(C)}(x)$, alors :

$$\mathcal{C}_{K(x)} = \{\pi(1), \pi(2), \dots, \pi(K)\}$$

2.2. Exemple pratique

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier

# Supposons un modèle entraîné qui prédit des probabilités
def top_k_prediction(probabilities, k):
    """
    Retourne les k classes les plus probables
    """
    # Indices triés par probabilité décroissante
    sorted_indices = np.argsort(probabilities)[::-1]
    return sorted_indices[:k]

# Exemple d'utilisation
proba = np.array([0.1, 0.6, 0.05, 0.2, 0.05]) # 5 classes
top_3 = top_k_prediction(proba, 3)
print(f"Top-3 classes: {top_3}") # [1, 3, 0]
print(f"Probabilités: {proba[top_3]}") # [0.6, 0.2, 0.1]
```

2.3. Métriques d'évaluation

Pour évaluer les prédictions Top-K, on utilise l'**accuracy@K** :

Définition 6.2 (Accuracy@K)

L'accuracy@K mesure la proportion d'exemples pour lesquels la vraie classe fait partie des K prédictions :

$$\text{Acc}@K = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[y_i \in \mathcal{C}_K(x_i)]$$

Propriétés :

- $\text{Acc}@1 \leq \text{Acc}@2 \leq \dots \leq \text{Acc}@C = 1$
- Plus K est grand, plus l'accuracy augmente mais l'ensemble devient moins informatif

3. Choix automatique de K

3.1. Problématique

Comment choisir K de manière **principielle** ? Deux approches principales :

1. **K fixe** : Tous les exemples ont des ensembles de taille K
2. **K adaptatif** : La taille varie selon la confiance du modèle

3.2. Méthode par seuillage de performance

Algorithme 6.1 (Choix de K par validation)

Entrée : Jeu de validation $\mathcal{D}_{\text{val}}$, seuil d'accuracy désiré τ

1. **Pour** $k = 1, 2, \dots, C$:
 - Calculer $\text{Acc}@k$ sur $\mathcal{D}_{\text{val}}$
2. **Retourner** : $K^* = \arg \min_k \{\text{Acc}@k \geq \tau\}$

Intuition : Choisir le plus petit ensemble qui garantit le niveau de performance souhaité.

3.3. Implémentation pratique

```
def find_optimal_k(y_true, y_proba, target_accuracy=0.95):
    """
    Trouve le K optimal pour atteindre l'accuracy cible
    """
    n_classes = y_proba.shape[1]
    accuracies = []

    for k in range(1, n_classes + 1):
        # Calculer l'accuracy@k
        correct = 0
        for i, (true_label, probas) in enumerate(zip(y_true, y_proba)):
            top_k = top_k_prediction(probas, k)
            if true_label in top_k:
                correct += 1

        acc_at_k = correct / len(y_true)
        accuracies.append(acc_at_k)

        # Retourner dès qu'on atteint la cible
        if acc_at_k >= target_accuracy:
            return k, accuracies

    return n_classes, accuracies # Fallback
```

Exercice 6.1

1. Implémentez une fonction qui calcule l'accuracy@K pour différentes valeurs de K
2. Sur un jeu de données de classification, tracez la courbe accuracy@K en fonction de K
3. Trouvez le K optimal pour atteindre 90% d'accuracy de couverture

4. Prédiction conformelle

4.1. Cadre théorique général

La prédiction conformelle offre un cadre **rigoureux** et **non-paramétrique** pour construire des ensembles de prédiction avec des garanties théoriques.

Définition 6.3 (Score de conformité)

Un **score de conformité** est une fonction $s : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ qui mesure à quel point une paire (x, y) est « conforme » au modèle appris.

Intuition : Plus le score est **faible**, plus (x, y) est cohérent avec les données d'entraînement.

Nous verrons plus bas des exemples de scores de conformité en fonction de la tâche à résoudre.

4.2. Algorithme de prédiction conformelle

Algorithme 6.2 (Prédiction conformelle)

Données : Entraînement $\mathcal{D}_{\text{train}}$, calibration $\mathcal{D}_{\text{cal}} = \{(X_i, Y_i)\}_{i=1}^n$

Étape 1 : Entraîner un modèle $\hat{f}$ sur $\mathcal{D}_{\text{train}}$

Étape 2 : Calculer les scores de conformité sur $\mathcal{D}_{\text{cal}}$:

$$S_i = s(X_i, Y_i) \quad \text{pour } i = 1, \dots, n$$

Étape 3 : Pour un nouvel exemple x , l'ensemble de prédiction est :

$$\mathcal{C}(x) = \{y \in \mathcal{Y} : s(x, y) \leq q\}$$

où q est le quantile d'ordre $\frac{[(n+1)(1-\alpha)]}{n}$ des scores $\{S_i\}$.

4.3. Garantie théorique fondamentale

Théorème 6.1 (Garantie de couverture)

Sous l'hypothèse d'échangeabilité (hypothèse moins forte que i.i.d.) des données $(X_1, Y_1), \dots, (X_n, Y_n), (X, Y)$, la prédiction conformelle garantit :

$$\mathbb{P}(Y \in \mathcal{C}(X)) \geq 1 - \alpha$$

Cette garantie est **exacte** en échantillon fini et **sans hypothèse paramétrique** sur le modèle sous-jacent.

Par exemple, dans une série temporelle avec dépendance temporelle, les données ne sont pas i.i.d. mais peuvent rester échangeables si l'on peut permuer l'ordre des observations sans changer la distribution jointe.

4.4. Score de conformité : rang de la prédiction

Un choix naturel de score de conformité en classification est le **rang** de la vraie classe :

Score de rang

$s(x, y) = \text{rg de la classe } y \text{ dans le classement par proba décroissantes}$

Propriété remarquable : Avec ce score, la prédiction conformelle redonne exactement le Top-K validé !

Si q est le quantile choisi, alors $\mathcal{C}(x)$ contient les classes de rang $\leq q$.

4.5. Implémentation du score de rang

```
def conformity_score_rank(probabilities, true_label):
    """
    Score de conformité basé sur le rang de la vraie classe
    """
    # Trier les probabilités par ordre décroissant
    sorted_indices = np.argsort(probabilities)[::-1]

    # Trouver le rang de la vraie classe (1-indexé)
    rank = np.where(sorted_indices == true_label)[0][0] + 1
    return rank


def conformal_prediction_rank(cal_probas, cal_labels, test_probas, alpha=0.05):
    """
    Prédiction conformelle avec score de rang
    """
    # Calculer les scores sur l'ensemble de calibration
    scores = [conformity_score_rank(prob, label)
               for prob, label in zip(cal_probas, cal_labels)]

    # Calculer le quantile
    n = len(scores)
    q_level = np.ceil((n + 1) * (1 - alpha)) / n
    quantile = np.quantile(scores, q_level)

    # Prédiction : classes de rang <= quantile
    predictions = []
    for prob in test_probas:
        sorted_indices = np.argsort(prob)[::-1]
        k = int(quantile)
        predictions.append(sorted_indices[:k])

    return predictions
```


5. Scores de conformité probabilistes

5.1. Score basé sur 1 - probabilité

Un score plus raffiné utilise directement les probabilités prédites :

Score probabiliste simple

$$s(x, y) = 1 - \hat{p}_y(x)$$

Intuition : Plus la probabilité de la vraie classe est élevée, plus le score est faible (donc plus conforme).

5.2. Score basé sur le produit de probabilités

Pour favoriser les cas où **plusieurs** classes ont des probabilités élevées :

Score de produit cumulé

Pour prédire un ensemble $\mathcal{S}$, le score peut être défini comme :

$$s(x, \mathcal{S}) = 1 - \prod_{y \in \mathcal{S}} \hat{p}_y(x)$$

ou alternativement, en utilisant la probabilité cumulative :

$$s(x, y) = 1 - \sum_{(j: \hat{p}_j(x) \geq \hat{p}_y(x))} \hat{p}_j(x)$$

5.3. Comparaison des scores

```
def score_1_minus_proba(probabilities, true_label):
    """Score : 1 - probabilité de la vraie classe"""
    return 1 - probabilities[true_label]

def score_cumulative_proba(probabilities, true_label):
    """Score : 1 - somme des probas des classes au moins aussi probables"""
    true_proba = probabilities[true_label]
    # Probabilité cumulative des classes >= true_proba
    cumulative = np.sum(probabilities[probabilities >= true_proba])
    return 1 - cumulative

def conformal_prediction_proba(cal_probas, cal_labels, test_probas,
                               score_func, alpha=0.05):
    """
    Prédiction conformelle avec score probabiliste
    """
    # Scores de calibration
    scores = [score_func(prob, label)
              for prob, label in zip(cal_probas, cal_labels)]

    # Quantile
    n = len(scores)
    q_level = np.ceil((n + 1) * (1 - alpha)) / n
    quantile = np.quantile(scores, q_level)

    # Prédictions
    predictions = []
    for prob in test_probas:
        pred_set = []
        for class_idx in range(len(prob)):
            if score_func(prob, class_idx) <= quantile:
                pred_set.append(class_idx)
        predictions.append(pred_set)

    return predictions
```

Exercice 6.2

1. Implémentez et comparez les différents scores de conformité (rang, 1-proba, cumulatif)
2. Analysez leur effet sur la taille des ensembles de prédiction
3. Vérifiez empiriquement la garantie de couverture $1 - \alpha$

6. Application à la régression

6.1. Principe général

En régression, l'objectif est de construire des **intervalles de prédiction** $[\hat{y}_{\text{low}}(x), \hat{y}_{\text{high}}(x)]$ avec garantie de couverture.

Objectif en régression conformelle

Construire $\mathcal{C}(x) = [a(x), b(x)]$ tel que :

$$\mathbb{P}(Y \in [a(X), b(X)]) \geq 1 - \alpha$$

L'intervalle doit être le plus **petit possible** tout en respectant la contrainte de couverture.

6.2. Intervalles de largeur constante

L'approche la plus simple utilise des intervalles symétriques autour de la prédiction :

Score de conformité constant

$$s(x, y) = |y - \hat{f}(x)|$$

L'ensemble de prédiction devient :

$$\mathcal{C}(x) = [\hat{f}(x) - q, \hat{f}(x) + q]$$

où q est le quantile des résidus absolus sur l'ensemble de calibration.

Exercice : Retrouver l'intervalle de prédiction à partir du score de conformité

6.3. Implémentation des intervalles constants

```
def conformal_regression_constant(cal_predictions, cal_true,
                                test_predictions, alpha=0.05):
    """
    Intervalles de prédiction de largeur constante
    """
    # Scores de conformité : résidus absolus
    residuals = np.abs(cal_true - cal_predictions)

    # Quantile pour la largeur d'intervalle
    n = len(residuals)
    q_level = np.ceil((n + 1) * (1 - alpha)) / n
    interval_width = np.quantile(residuals, q_level)

    # Construction des intervalles
    lower_bounds = test_predictions - interval_width
    upper_bounds = test_predictions + interval_width

    return lower_bounds, upper_bounds

# Exemple d'usage
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# ... entraînement du modèle ...
cal_preds = model.predict(X_cal)
test_preds = model.predict(X_test)

# Intervalles avec 95% de couverture
lower, upper = conformal_regression_constant(
    cal_preds, y_cal, test_preds, alpha=0.05
)
```

6.4. Intervalles adaptatifs

Pour des intervalles plus informatifs, on peut adapter leur largeur selon l'**incertitude locale** du modèle :

Score de conformité adaptatif

$$s(x, y) = \frac{|y - \hat{f}(x)|}{\sigma(x) + \varepsilon}$$

où $\sigma(x)$ est une estimation de l'incertitude locale (ex: écart-type des prédictions d'un ensemble de modèles) et $\varepsilon > 0$ évite la division par zéro.

Avantage : Intervalles plus étroits dans les régions de forte confiance, plus larges ailleurs.

Exercice : Retrouver l'intervalle de prédiction à partir de $s(x, y)$

6.5. Estimation de l'incertitude locale

Plusieurs approches pour estimer $\sigma(x)$:

1. **Bootstrap** : Écart-type des prédictions sur différents échantillons bootstrap
2. **Bagging** : Variance des prédictions d'un ensemble de modèles
3. **Quantile regression** : Différence entre quantiles prédits
4. **Réseaux bayésiens** : Variance a posteriori

6.6. Évaluation des intervalles de prédiction

Métriques d'évaluation

1. **Taux de couverture empirique :**

$$\frac{1}{n} \sum_{i=1}^n \mathbb{1}[y_i \in [a(x_i), b(x_i)]]$$

2. **Largeur moyenne des intervalles :**

$$\frac{1}{n} \sum_{i=1}^n (b(x_i) - a(x_i))$$

3. **Efficacité conditionnelle :** Vérifier que la couverture ne dépend pas trop des valeurs de X

Exercice 6.3

1. Comparez les intervalles constants vs adaptatifs sur un jeu de données avec hétéroscédasticité
2. Vérifiez empiriquement la garantie de couverture pour différents niveaux α
3. Analysez l'efficacité (largeur d'intervalle) des deux approches

Rappel : il y a hétéroscédasticité lorsque la variance des résidus conditionnellement aux variables explicatives n'est pas constante et homoscedasticité dans le cas contraire.

7. Synthèse et perspectives

7.1. Tableau comparatif des approches

Méthode	Garanties	Flexibilité	Complexité
Top-K fixe	Aucune	Faible	Très simple
Top-K adaptatif	Empirique	Moyenne	Simple
Conformal (rang)	Théorique	Moyenne	Moyenne
Conformal (proba)	Théorique	Élevée	Moyenne
Intervalles constants	Théorique	Faible	Simple
Intervalles adaptatifs	Théorique	Élevée	Complexe

7.2. Avantages de la prédiction conformelle

Points forts

- ✓ **Garanties théoriques** : Couverture exacte en échantillon fini
- ✓ **Model-agnostic** : Fonctionne avec n'importe quel modèle sous-jacent
- ✓ **Non-paramétrique** : Aucune hypothèse distributionnelle
- ✓ **Computational efficace** : Pas de réentraînement nécessaire
- ✓ **Robustesse** : Marche même si le modèle de base est mal spécifié

7.3. Limitations et défis

Limitations

- × **Hypothèse d'échangeabilité** : Peut être violée (dérive temporelle, shift)
- × **Efficacité marginale** : Garantie sur la population moyenne, pas localement
- × **Taille d'ensemble** : Peut devenir très grande si le modèle est peu performant
- × **Calibration** potentiellement **nécessaire** : Besoin d'un jeu de données dédié

8. Conclusion

La prédiction d'ensembles et la quantification d'incertitude représentent un paradigme essentiel pour rendre l'apprentissage automatique plus **fiable** et **interprétable**.